문제 1 (유사 문제) — 조합회로 최대 지연

아래의 논리 회로가 있다. 입력은 A, B, C, D, E이며, 각 게이트의 지연은 다음과 같다.

- NOT 게이트: 1 단위
- AND/OR 게이트 (2-input): 2 단위
- AND/OR 게이트 (3-input): 3 단위

회로 구성:

- Level 1: AND2(A, B) → P, OR2(C, D) → Q, NOT(E) → E'
- Level 2: AND3(P, Q, E') → R, AND2(B, C) → S
- Level 3: OR2(R, S) → T, NOT(D) → D'
- Level 4: AND3(T, D', E) → **출력 F**

a. 최대 지연을 구하여라.

- i. 모든 입력은 보수와 비보수 모두 가능하다고 가정한다.
- ii. 비보수의 입력만 가능하고, 보수화를 위해 NOT 게이트를 추가해야 한다.

b. 입력 C부터 출력 F까지의 최대 지연을 구하여라. (모든 입력은 보수/비보수 모두 가능)

✅ 문제 1 — 조합회로 최대 지연

🔹 (a-i) 보수/비보수 모두 사용 가능

👉 각 입력에서 출력 F까지 **가장 긴 경로(critical path)** 찾기

경로 후보 분석

예를 들어 가장 긴 경로:

A → P → R → T → F

- AND2(A,B) → P = 2
- AND3(P,Q,E') → R = 3
- OR2(R,S) → T = 2
- AND3(T,D',E) → F = 3

👉 총 지연:

$$2 + 3 + 2 + 3 = 10$$

다른 경로도 비교:

E → NOT → R → T → F

- NOT(E) = 1
- AND3 = 3
- OR2 = 2
- AND3 = 3
  👉 총: **9**

✅ **정답**

👉 **최대 지연 = 10**

🔹 (a-ii) 보수 입력 없음 (NOT 추가 필요)

추가되는 NOT:

- E → E' : +1
- D → D' : +1

- D → D' : +1

👉 기존 critical path에 영향 확인

기존 경로:
A → P → R → T → F (NOT 없음)

👉 여전히 최장

✅ 정답

👉 **최대 지연 = 10 (변화 없음)**

🔹 (b) 입력 C → 출력 F

경로:

C → Q → R → T → F

- OR2(C,D) = 2
- AND3 = 3
- OR2 = 2
- AND3 = 3

- AND3 = 3

👉 총:

$$2 + 3 + 2 + 3 = 10$$

✅ 정답

👉 **C → F 최대 지연 = 10**

문제 3 (유사 문제) — n비트 1의 보수 및 증감 회로

n비트 숫자의 1의 보수를 계산한 후 1을 빼는 회로를 n개의 모듈로 구현하려 한다. 각 모듈은 해당 비트의 보수를 취한 다음, 이전 단계에서 오는 **borrow(빌림수)**를 처리한다. 가장 오른쪽 모듈의 초기 borrow 입력은 **1**이다.

a. NAND 게이트만 사용하여 각 모듈의 블록도를 설계하여라 (특히 오른쪽 첫 번째 모듈을 상세히 설계).

b. n비트에 대한 **총 지연**을 계산하여라.

c. 한 번에 **2비트씩 처리**하도록 회로를 재설계하여 속도를 향상시켜라. NAND 게이트 회로를 보이고 총 지연도 계산하여라.

✅ 문제 3 — 1의 보수 후 -1 회로

🔹 핵심 개념

1의 보수 → NOT
그 다음 **-1 = borrow 발생 구조**

👉 즉, **Full Subtractor 구조** 필요

🔹 (a) NAND만 사용

Full Subtractor 기본식:

- Difference:

$$D = A \oplus B \oplus Bin$$

- Borrow:

$$Bout = \overline{A}B + Bin(\overline{A \oplus B})$$

👉 XOR, AND, OR → 전부 NAND로 변환

핵심:

- NOT: NAND(A,A)

- AND: NAND → NAND
- OR: 드모르간

👉 시험 포인트:
"XOR를 NAND 4~5개로 구현"

🔹 (b) 총 지연

👉 ripple 구조 (borrow가 오른쪽 → 왼쪽 전달)

각 단계 지연 = t

👉 총 지연:

$$T = n \cdot t$$

✅ **정답**

👉 **O(n)**

🔹 (c) 2비트씩 처리

👉 carry-lookahead 느낌

- 2비트 블록 병렬 처리
- borrow propagation 감소

👉 총 지연:

$$T = \frac{n}{2} \cdot t$$

✅ **정답**

👉 **지연 절반으로 감소**

문제 7 (유사 문제) — 디코더를 이용한 논리함수 구현

3-to-8 **Active high 출력 디코더** (입력: a, b, c / 출력: m0~m7)를 사용하는 아래 회로가 있다.

- 출력 X: m0, m2, m4, m6을 OR 게이트로 연결
- 출력 Y: m1, m2, m5, m7을 OR 게이트로 연결

입력 a, b, c에 대한 X와 Y의 진리표를 작성하여라.

✅ 문제 7 — 디코더

3-to-8 디코더 → minterm

a b c	m
000	m0
001	m1
010	m2
011	m3
100	m4
101	m5
110	m6
111	m7

🔹 X = m0 + m2 + m4 + m6

👉 짝수 minterm

👉 특징:

$$c = 0$$

🔹 Y = m1 + m2 + m5 + m7

직접 계산:

a b c	Y
000	0
001	1
010	1
011	0
100	0
101	
110	0
111	1

✅ 정답 요약

- **X = c'**
- **Y = Σ(1,2,5,7)**

문제 12 (유사 문제) — 멀티플렉서를 이용한 함수 구현

2갈래(2-to-1) 멀티플렉서를 사용하여 다음 함수를 구현하여라.

$$f(x, y, z) = \Sigma m(0, 2, 3, 5, 6)$$

✅ 문제 12 — MUX 구현

함수:

$$f(x, y, z) = \Sigma(0, 2, 3, 5, 6)$$

🔹 방법

👉 2-to-1 MUX → 선택선 1개

보통:

- x = select

🔹 분리

x = 0:

minterm: 0,2,3
→ f = 1 (y,z 조합 거의 대부분)

x = 1:

minterm: 5,6
→ 경우 따로 계산

🔹 최종 형태

$$f = x' f_0 + x f_1$$

👉 MUX 구조 완성